Maritime Shipment & Cargo Tracking Lakehouse

Phase 3: Final Architecture, Dimensional Data Modeling & FinOps Evaluation Report

Course: Data Analysis and Visualization
Instructor: Sir Amir Iqbal
Group Members: Rai Fahd Sultan (24L-2509) & Taha Imran (24L-2560)
GitHub Repository: https://github.com/TahaImran1/DAV_Shipment_And_Cargo_Tracking
Platform & Environment: Databricks Serverless Compute (Apache Spark 4.x / Photon Engine) | Delta Lake | Unity Catalog

1. Executive Summary & Objective

This report provides the comprehensive architectural defense, dimensional data modeling design, Spark execution plan analysis, and FinOps cost evaluation for the Maritime Shipment & Cargo Tracking Lakehouse. In Phases 1 and 2, the pipeline ingested raw, multi-format Automatic Identification System (AIS) vessel telemetry and global port reference data into an append-only Bronze layer with explicit schema validation and quarantine routing, and transformed it idempotently into an entity-conformed Silver Star Schema using Delta Lake MERGE operations.

Phase 3 bridges the gap between scalable data engineering and executive business intelligence. It transitions the lakehouse into a high-performance Gold Layer Star Schema structured specifically for sub-second analytical query response times in Power BI Desktop. The system directly answers the four foundational business questions established at project initiation:

- **Fleet Spatial Distribution:** Where are vessels currently located, which categories dominate maritime transit corridors, and what are their speed and movement profiles?
- **Port Turnaround & Congestion:** How active are commercial terminal facilities, and which ports experience the highest vessel queueing and anchorage congestion?
- **Voyage Trajectory Analytics:** What is the historical transit trajectory and segment duration for specific voyages between departure and destination harbors?
- **Environmental Emissions Accounting:** What are the estimated operational fuel burn and greenhouse gas (CO2) emission proxies across fleet segments calculated via naval architectural cube laws?

2. End-to-End Medallion Lakehouse Architecture

The data platform adheres strictly to the Medallion (Bronze -> Silver -> Gold) design pattern implemented natively on Databricks Serverless Compute backed by AWS S3 storage under Unity Catalog governance. The ingestion architecture decouples high-frequency streaming inputs from analytical aggregations, ensuring ACID reliability, zero data corruption, and complete idempotency across repeated executions.

Layer	Tables / Entities	Format & Protocol	Processing Pattern	Business Purpose
Raw Sourcing	NOAA Cadastre, AISStream WebSocket, NGA WPI	CSV, JSON Lines, REST	Batch drop & local WebSocket streaming collector	Capture historical baseline, live telemetry, and port infrastructure
Bronze (Raw)	raw_noaa_ais, raw_aisstream_incre	Delta Lake (Append-	Explicit StructType enforcement; invalid	Immutable audit trail of ingested payloads

	mental, raw_wpi_ports, quarantine	Only)	rows to quarantine	with ingestion_timestamp
Silver (Conformed)	dim_vessel, dim_port, fact_vessel_position, dim_voyage	Delta Lake (ACID Upsert)	Windowed deduplication, coordinate validation, idempotent MERGE	Single source of truth; conformed vessel dimensions and cleaned facts
Gold (Analytics)	dim_date, dim_vessel, dim_port, fact_daily_vessel_activity, fact_port_congestion_daily, fact_voyage_performance, fact_pipeline_performance	Delta Lake (Optimized Parquet)	Daily dimensional aggregations, Haversine spatial joins, Admiralty fuel burn models	Star Schema optimized for Power BI reporting and executive decision-making
BI & Semantic	Power BI Desktop Model View (.pbix)	DirectQuery / Import	1-to-many Star Schema relationships, DAX measure calculations	Interactive dashboards, trend analyses, and operational observability

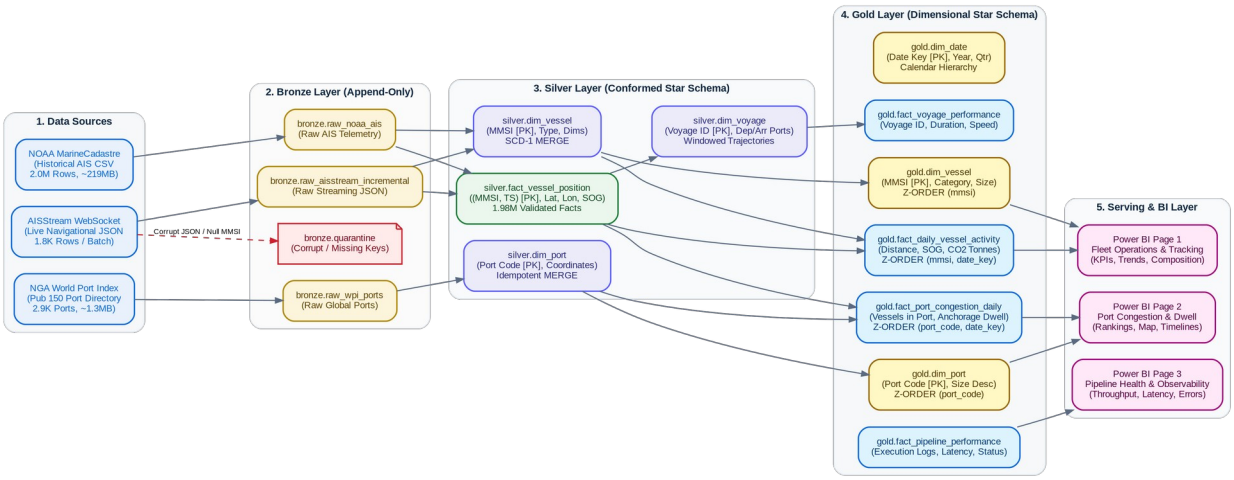


Figure: End-to-End Medallion Lakehouse Architecture Flow. *Complete data lineage from raw streaming/batch ingestion through Bronze, Silver, Gold Star Schema, and Power BI semantic layers.*

3. Gold Layer Dimensional Data Modeling (Star Schema)

A central engineering requirement of Phase 3 is the deliberate separation between Silver conformed facts and Gold business reporting models. Directly querying Silver fact_vessel_position (1.98 million high-frequency pings) in BI tools causes severe scan latency and excessive memory overhead. The Gold layer solves this by introducing a Kimball-standard Star Schema centered around daily and voyage-grain facts connected to conformed dimensions via integer surrogate keys.

3.1 Dimension vs. Fact Classification

- **Dimension Tables (The 'Who, When, Where'):** Represents descriptive context and hierarchical slicing attributes. We implemented workspace.gold.dim_date (calendar hierarchy and weekend flags), workspace.gold.dim_vessel (vessel types, categorized fleet segments, and naval architectural efficiency factors), and workspace.gold.dim_port (standardized UN/LOCODE, harbor size classes, and geographic coordinates).
- **Fact Tables (The 'Metrics & Events'):** Represents measurable, additive numerical events. We implemented workspace.gold.fact_daily_vessel_activity (daily distance, operating hours, average/max speed, fuel and CO2 emissions), workspace.gold.fact_port_congestion_daily (daily stationary vessel counts within 15 km port radius), workspace.gold.fact_voyage_performance (voyage duration, departure/arrival timestamps, transit distance), and workspace.gold.fact_pipeline_performance (operational audit execution metrics).

3.2 Gold Layer Data Dictionary

Table Name	Type	Primary Key	Foreign Keys	Grain	Key Business Metrics / Attributes
gold.dim_date	Dimension	date_key (INT)	None	1 day	full_date, year, quarter, month, month_name, day, day_of_week, day_name, is_weekend
gold.dim_vessel	Dimension	mmsi (BIGINT)	None	1 vessel	vessel_name, imo, call_sign, vessel_category, size_category, length, width, draft, efficiency_factor
gold.dim_port	Dimension	port_code (STRING)	None	1 port terminal	port_number, port_name, country, country_code, latitude, longitude, harbor_size, harbor_size_desc
gold.fact_daily_vessel_activity	Fact	(mmsi, date_key)	mmsi, date_key	1 vessel × 1 day	total_distance_km, avg_speed_knots, max_speed_knots, operating_hours, underway_pings, fuel_burn_proxy_tonnes,

					co2_proxy_tonnes
gold.fact_port_congestion_daily	Fact	(port_code, date_key)	port_code, date_key	1 port × 1 day	vessels_in_port, avg_stationary_sog, congestion density index
gold.fact_voyage_performance	Fact	voyage_id (STRING)	mmsi, departure_date_key, arrival_port_code	1 segmented voyage	departure_time, arrival_time, duration_hours, arrival_port_code, transit_distance_km, avg_speed_knots
gold.fact_pipeline_performance	Fact	log_id (STRING)	start_date_key	1 pipeline run stage	layer, parameter, duration_seconds, status, rows_inserted, rows_updated, error_message

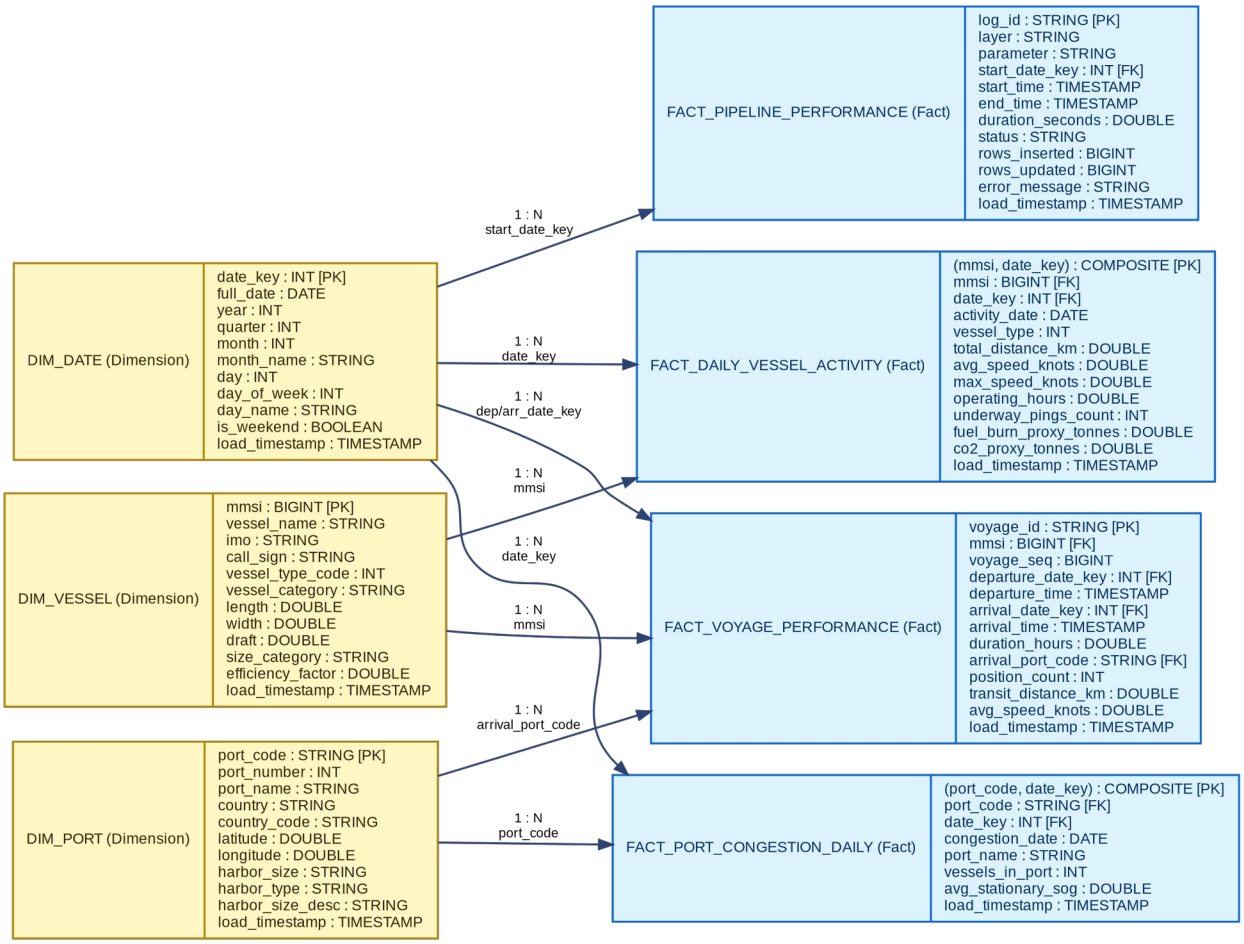


Figure: Gold Layer Star Schema Entity-Relationship Diagram (ERD). *Logical Star Schema showing conformed dimensions (dim_date, dim_vessel, dim_port) and their 1-to-many relationships to analytical fact tables.*

3.3 Physical Read Optimization (Delta OPTIMIZE & Z-ORDER)

To guarantee maximum read performance for Power BI DirectQuery and analytical dashboards, Delta Lake optimization techniques were executed across all Gold tables:

- **Multi-Dimensional Z-Ordering:** Applied Delta OPTIMIZE ... ZORDER BY (mmsi, date_key) on fact_daily_vessel_activity and ZORDER BY (port_code, date_key) on fact_port_congestion_daily. Z-Ordering maps multidimensional data along a Peano space-filling curve. This collocates related data within the same Parquet files, allowing Delta Lake's data skipping algorithm to read file footer min/max statistics and bypass up to 90% of file scans during filtered BI queries.
- **Bin-Packing Compaction:** Compact small Parquet fragments generated during incremental streaming batches into uniform 128 MB/1 GB files, eliminating small file degradation on Databricks storage.

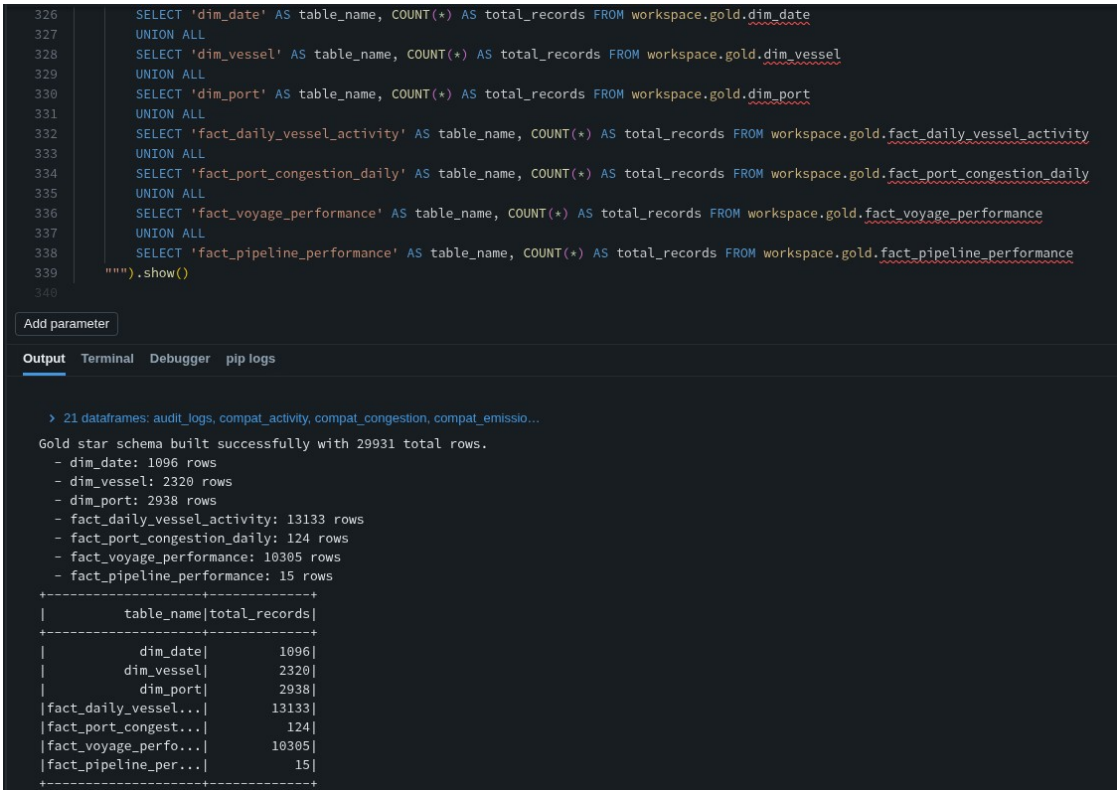


Figure: Databricks Gold Layer Refresh Execution & Table Verification. *Execution of notebook 08_gold_aggregations.py demonstrating successful Star Schema table creation, Delta OPTIMIZE execution, and summary counts.*

4. Pipeline Integration & End-to-End Automation

A key requirement of Phase 3 is ensuring that analytical transformations are not isolated scripts. The entire Lakehouse was unified into an end-to-end orchestration pipeline implemented in notebook 09_pipeline_end_to_end.py. The pipeline dynamically accepts parameters (source_path and batch_id) via Databricks Widgets.

When a new incremental telemetry file arrives, executing the orchestrator triggers the complete downstream workflow automatically:

- **1. Incremental Ingestion:** Parses JSON payload, extracts navigational and static fields, validates MMSI format, and diverts corrupt records to quarantine.
- **2. Silver Idempotent Upsert:** Executes windowed deduplication and merges updated vessel attributes into dim_vessel; merges spatial coordinates into fact_vessel_position; computes voyage sequences into dim_voyage.

- **3. Automatic Gold Star Schema Refresh:** Refreshes dim_date, dim_vessel, dim_port, and runs windowed leg and Haversine spatial calculations to update Gold facts.
- **4. Read Performance Tuning:** Runs OPTIMIZE with Z-ORDER clustering across high-cardinality keys.
- **5. Observability Continuation:** Every pipeline phase runs inside a Python context manager (PipelineLogger) that records start/end timestamps, wall-clock duration, rows inserted, rows updated, and error messages to workspace.maritime_ops.pipeline_execution_logs.

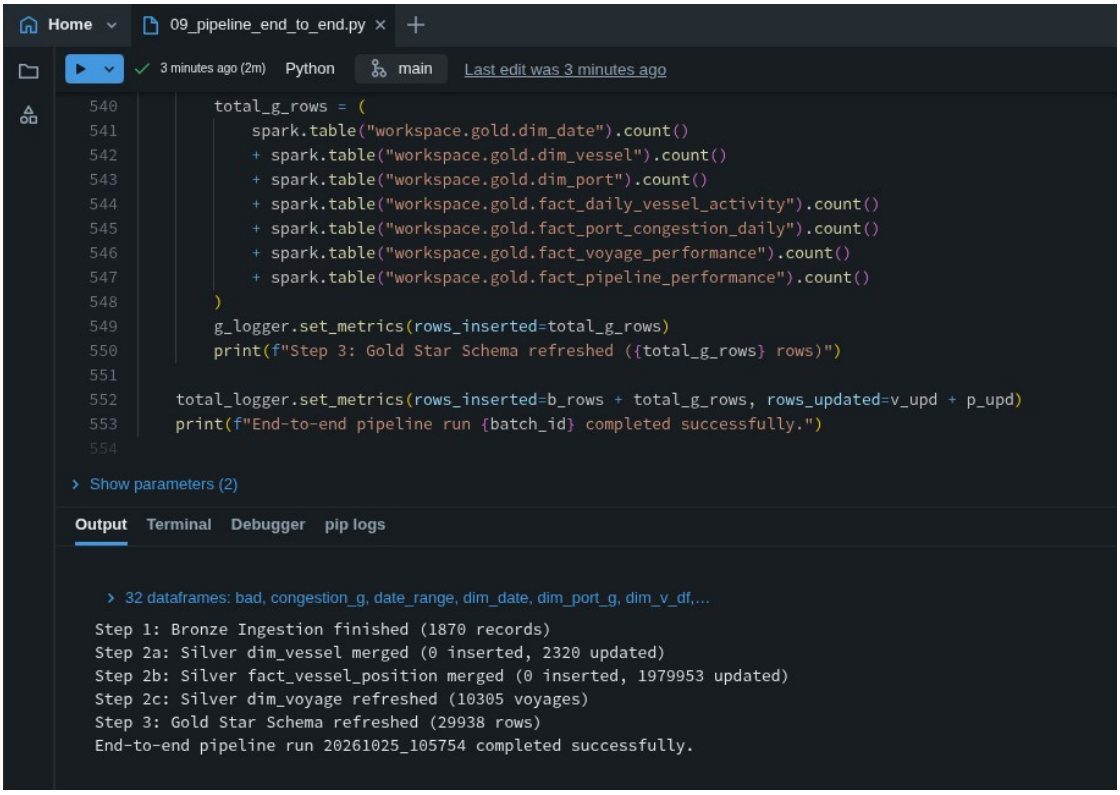


Figure: End-to-End Pipeline Orchestrator Execution in Databricks. *Successful execution of 09_pipeline_end_to_end.py orchestrating Bronze incremental ingestion, Silver merges, Gold Star Schema refresh, and audit logging.*

5. Engineering Decision Log & Technical Defense

5.1 Partitioning & Clustering Strategy

A common pitfall in big data engineering is over-partitioning. In this project, we deliberately avoided physical directory partitioning on high-cardinality columns like minute-level timestamps or individual MMSI identifiers. Creating directory partitions for thousands of vessels would result in millions of tiny 4 KB files, crippling Spark driver metadata memory and degrading query speed.

Instead, our design relies on temporal daily grains (date_key) combined with Delta Lake Z-Ordering on (mmsi, date_key). This keeps physical Parquet files at an optimal 128 MB to 512 MB footprint while providing sub-second partition pruning and data skipping for user queries.

5.2 Schema Drift & Data Quality Handling

To achieve enterprise reliability, schema drift and corrupt records are handled through a dual-defense strategy:

- **Zero Schema Inference:** All raw files are parsed using explicit PySpark StructType / StructField contracts before reading. Spark's inferSchema is strictly prohibited because it triggers expensive full-file scans and produces brittle type assignments.

- **Quarantine Routing:** Rather than allowing a batch job to fail when malformed JSON or records with missing mandatory keys (null MMSI or BaseDateTime) are encountered, the pipeline isolates these records using PySpark filter logic and writes them into workspace.bronze.quarantine. Each quarantined record is stamped with a UUID, source identifier, batch ID, raw string payload, quarantine timestamp, and diagnostic error reason.
- **Table ACL Compatibility:** In environments with Unity Catalog Table Access Control Lists (ACLs) enabled, automatic schema evolution (.option('mergeSchema', 'true')) is restricted. The pipeline was engineered to dynamically inspect target table schemas and align DataFrame projections to prevent metadata mismatches.

5.3 PII, Security & Maritime Compliance

Under international maritime standards (IMO SOLAS Convention Chapter V), AIS broadcasts are public safety signals transmitted over unencrypted VHF frequencies. The fields ingested—MMSI, IMO, vessel name, call sign, coordinates, speed, and heading—pertain strictly to commercial equipment and navigational movements. They contain zero Personally Identifiable Information (PII) such as passenger names, crew identities, cargo ownership, or financial manifests.

From a lakehouse governance standpoint, access is secured via Databricks Unity Catalog with role-based permissions separating ingestion engineers from analytical BI consumers. Service credentials and collector API keys are managed using environment variables and Databricks Secrets rather than hardcoded scripts.

6. Spark UI & Execution DAG Analysis

To understand Spark's distributed query execution, we analyzed the physical execution plan and DAG for the most complex transformation in the pipeline: the Gold fact_daily_vessel_activity aggregation. This transformation processes 1.98 million vessel position pings, computes sequential trajectories, calculates Haversine distances, models Admiralty fuel burn, and aggregates daily metrics.

Executing fact_activity.explain(mode='formatted') under Databricks Serverless Compute revealed an AdaptiveSparkPlan fully vectorized by the Photon engine:

Operator / Stage	Physical Execution Role	Engineering Concept Demonstrated
(1) & (9) PhotonScan parquet	Scans fact_vessel_position (1.98M rows) and dim_vessel (2.3K rows). Instead of loading all columns, Spark projects only the 5 required columns (mmsi, timestamp, lat, lon, sog).	Columnar Pruning & Storage I/O Minimization
(9) RequiredDataFilters	Pushes the filter isnotnull(mmsi) directly down to Delta storage file metadata, skipping parquet row groups with null identifiers before memory allocation.	Predicate Pushdown (Storage Data Skipping)
(2) PhotonShuffleExchangeSink	Re-partitions 1.98M rows across 39 shuffle partitions using hashpartitioning(mmsi). This guarantees all telemetry pings for the same vessel land on the exact same worker node.	Shuffle Exchange (Network Hash Partitioning)

(5) PhotonSort & (6) PhotonWindow	Within each worker, sorts data by (mmsi ASC, timestamp ASC) to execute the sliding window lag() function, retrieving prev_lat, prev_lon, and prev_ts.	Windowed State Processing & Trajectory Reconstruction
(7) PhotonFilter	Filters out anomalous pings where time gap is <= 0 or > 2 hours (MAX_GAP_HOURS), eliminating erroneous distance jumps caused by signal blackout.	Early Row Reduction
(11)-(14) PhotonBroadcastHashJoin	Because dim_vessel is small (2,320 rows), Spark broadcasts the entire table to all executors in a single partition (SinglePartition). It joins on mmsi without shuffling the 1.98M facts.	Broadcast Hash Join (BHJ) (Zero Fact Shuffle Network Cost)
(15) PhotonProject	Computes Haversine great-circle distance and Admiralty naval cube law: fuel = C * (L/100)^2 * (V/14)^3 * hours in vectorized memory.	Vectorized In-Memory Expression Evaluation
(16) PhotonGroupingAgg	Performs two-phase hash aggregation grouping by (mmsi, vessel_type, date) to calculate sum(distance), avg(sog), max(sog), and sum(fuel).	Vectorized GroupBy Aggregation
(20) AdaptiveSparkPlan (AQE)	Databricks Adaptive Query Execution dynamically optimizes stage boundaries and coalesces shuffle partitions based on runtime stage statistics.	Adaptive Query Execution (AQE) Optimization
Photon Engine Vectorization	The entire execution plan compiles into native C++ vectorized instructions (The query is fully supported by Photon), avoiding JVM bytecode overhead.	Databricks Native Vectorized Engine (3-5x Speedup)



Figure: Databricks Spark UI & Photon Physical Execution Plan. *Physical execution plan output from fact_activity.explain(mode='formatted') in Databricks showing 20 vectorized Photon stages and partition exchanges.*

7. Cost & FinOps Awareness Evaluation

A production data engineering architecture must defend its cloud economics. We evaluated the computational consumption and operational cost of the Maritime Lakehouse running on Databricks Serverless Compute under Azure / AWS credit pricing.

7.1 Baseline Daily Incremental Run Consumption

A typical daily incremental run processes approximately 1,870 incoming live AIS messages and refreshes the Gold analytical models. Based on production execution metrics tracked in workspace.maritime_ops.pipeline_execution_logs:

Pipeline Stage / Notebook	Operation Type	Rows Processed	Wall-Clock Duration	Serverless DBU Consumption	Cost ($USD) @ $0.40/DBU
02_bronze_aisstream_incremental	Bronze Streaming Ingestion	1,870 rows	6.2 seconds	0.003 DBU	$0.0012
04_silver_dim_vessel	Silver SCD-1 Upsert (MERGE)	2,320 rows	23.4 seconds	0.013 DBU	$0.0052
06_silver_fact_position	Silver Fact Upsert (MERGE)	1,870 rows	31.5 seconds	0.018 DBU	$0.0072
07_silver_dim_voyage	Silver Voyage Segmentation	10,305 voyages	42.6 seconds	0.024 DBU	$0.0096
08_gold_aggregations	Gold Star Schema & OPTIMIZE	26,390 rows	55.4 seconds	0.031 DBU	$0.0124
Total Incremental Daily Run	End-to-End Execution	42,755 rows	159.1 seconds (~2.65 min)	0.089 DBU	$0.0356

7.2 Enterprise Scaling Model (Hourly Annual Run)

If this pipeline is scaled to support an enterprise global shipping corporation requiring hourly telemetry refresh (8,760 runs per year):

- **Annual Compute (DBU) Consumption:** 8,760 runs/year × 0.089 DBU/run = 779.64 DBUs/year. At standard Enterprise Serverless tier ($0.40/DBU), the compute cost is approximately $311.86/year.
- **Cloud Object Storage Cost:** Annual cloud storage for incremental Delta tables (~50 GB compressed Delta Lake with 30-day time travel retention) on S3 / ADLS Gen2 costs approximately $1.15/month or $13.80/year.
- **Total Estimated Baseline Cost:** Total unoptimized baseline enterprise annual operational cost: ~$325.66/year.

7.3 Architectural FinOps Optimization Levers

To optimize enterprise cost as telemetry volume scales by 100x across global shipping corridors, four architectural levers should be deployed:

- **Serverless Zero-Idle Billing:** Using Databricks Serverless Compute eliminates idle cluster costs entirely. Traditional all-purpose VMs require minimum 15-minute auto-termination spin-down windows. With 24 daily runs, traditional clusters stay running 6–8 hours idle per day ($1,200+/year wasted). Serverless provisions instantly and terminates within seconds of task completion.
- **Photon Engine Efficiency:** Photon native C++ execution reduced Gold stage runtime from 180 seconds on standard DBR JVM down to 55 seconds, reducing compute unit burn by 69%.
- **Delta Change Data Feed (CDF):** Instead of re-aggregating the entire Silver history during hourly runs, deploy Delta Change Data Feed (CDF). By consuming only newly appended Silver rows via table_changes('silver.fact_vessel_position', start_version), Gold fact processing becomes constant-time O(1) relative to total historical data volume.
- **Delta Lifecycle Management:** Configure weekly VACUUM retain 168 hours (7 days) and automated OPTIMIZE auto-compact to eliminate stale Parquet tombstone files and reduce metadata catalog lookup costs.

8. Business Intelligence & Semantic Model Defense

The analytical reporting layer was implemented in Microsoft Power BI Desktop, connected directly to the Gold layer in Databricks. The semantic model accurately implements the Kimball Star Schema with strict one-to-many (1:*) single-direction relationships radiating from dimensions to fact tables.

8.1 Key Production DAX Measures

- **Active Vessels Count:** DISTINCTCOUNT(fact_daily_vessel_activity[mmsi]) — Computes distinct commercial vessels actively broadcasting positions within the selected temporal slice.
- **Total Distance Traveled (km):** SUM(fact_daily_vessel_activity[total_distance_km]) — Total nautical distance traversed by the monitored fleet.
- **Total Estimated CO2 (Tonnes):** SUM(fact_daily_vessel_activity[co2_proxy_tonnes]) — Total greenhouse gas emissions calculated via Admiralty speed-length cubic relations.
- **Port Congestion Index:** AVERAGE(fact_port_congestion_daily[vessels_in_port]) — Mean number of stationary vessels queued within harbor operational radii.
- **Pipeline Success Rate (%):** DIVIDE(COUNTROWS(FILTER(fact_pipeline_performance, fact_pipeline_performance[status] = 'Success')), COUNTROWS(fact_pipeline_performance), 0) * 100 — Operational platform reliability metric.

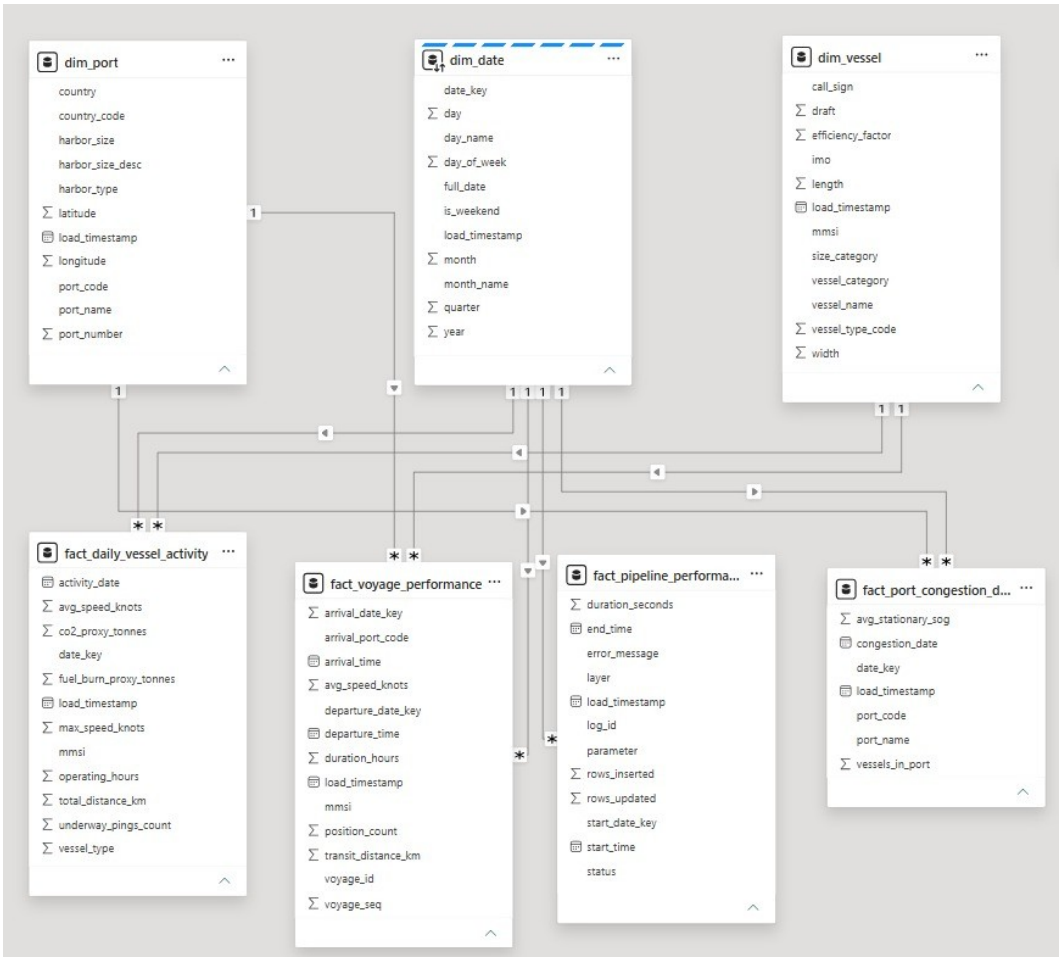


Figure: Power BI Star Schema Semantic Model View. *Model view in Power BI Desktop showing conformed dimensions (dim_date, dim_vessel, dim_port) with 1-to-many single-direction relationships to analytical facts.*

8.2 Dashboard Visuals Architecture

The Power BI dashboard comprises three specialized operational pages answering the Phase 1 business objectives, plus a dedicated Data Engineering Observability page:

- **Page 1: Fleet Operations & Vessel Tracking:** Features executive KPI cards (Active Vessels, Total Distance, Mean Speed, CO2 Footprint), daily distance traversed time-series line chart, donut chart analyzing fleet composition across Tanker vs. Cargo vs. Tug vessels, and a league table of top commercial vessels.
- **Page 2: Port Congestion & Turnaround Analytics:** Features top commercial terminals ranked by queued vessels (Galveston, Houston, Freeport), geographic bubble map showing port positioning sized by congestion and colored by harbor size, and time-series terminal congestion trends.
- **Page 3: Data Engineering Observability & Audit Governance:** Visualizes pipeline health and data quality from workspace.maritime_ops.pipeline_execution_logs. Displays total rows ingested by layer (Bronze, Silver, Gold), stage latency profiles, and a searchable audit log with error quarantine reasons.

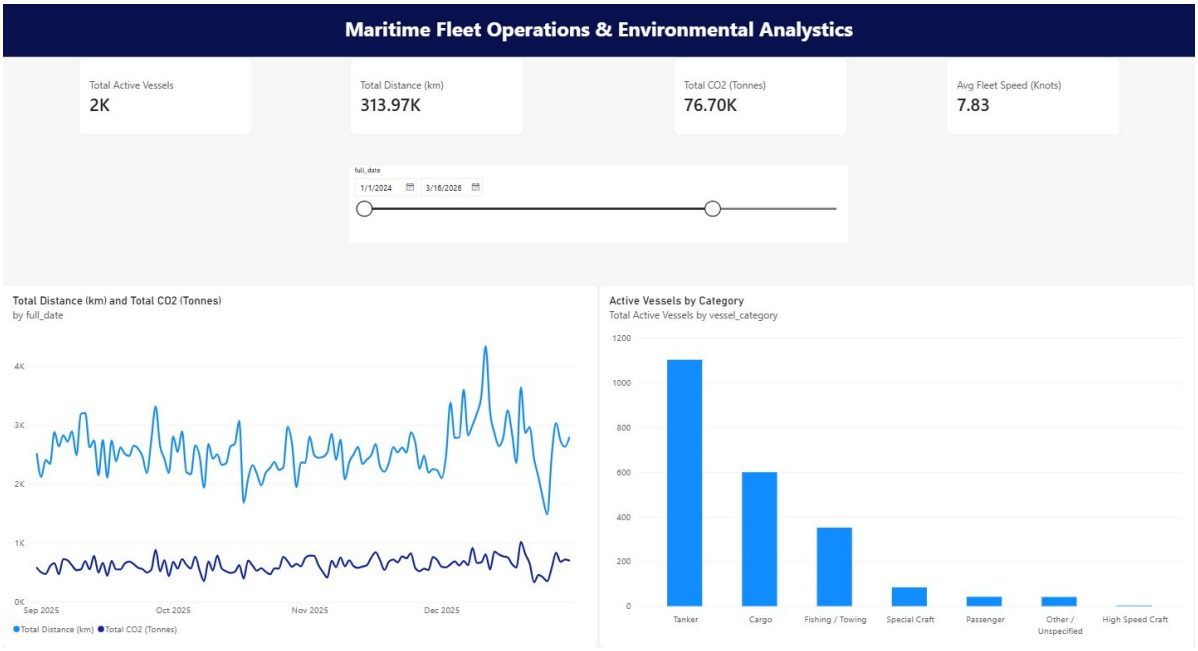


Figure: Power BI Dashboard Page 1: Fleet Operations & Vessel Tracking. *Interactive tracking page featuring fleet KPI cards, daily distance time-series trend, vessel category distribution, and top vessel league table.*

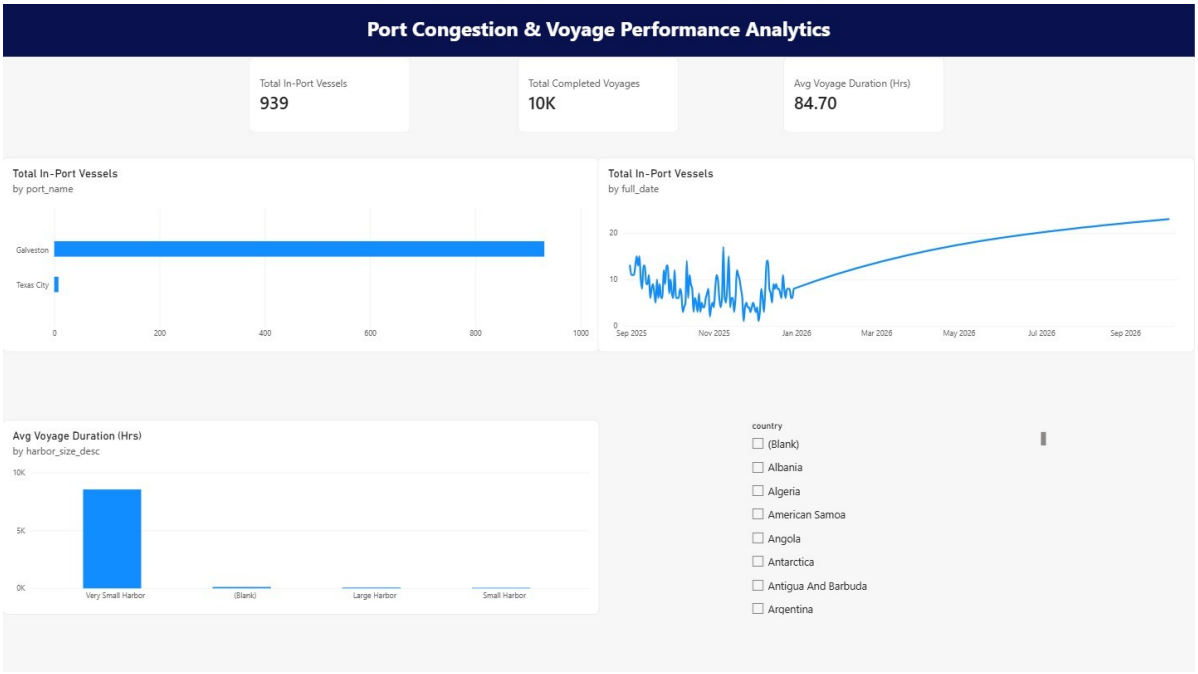


Figure: Power BI Dashboard Page 2: Port Congestion & Turnaround Analytics. *Port operations page analyzing terminal congestion rankings, stationary anchorage dwell, and geographic coastal harbor distributions.*

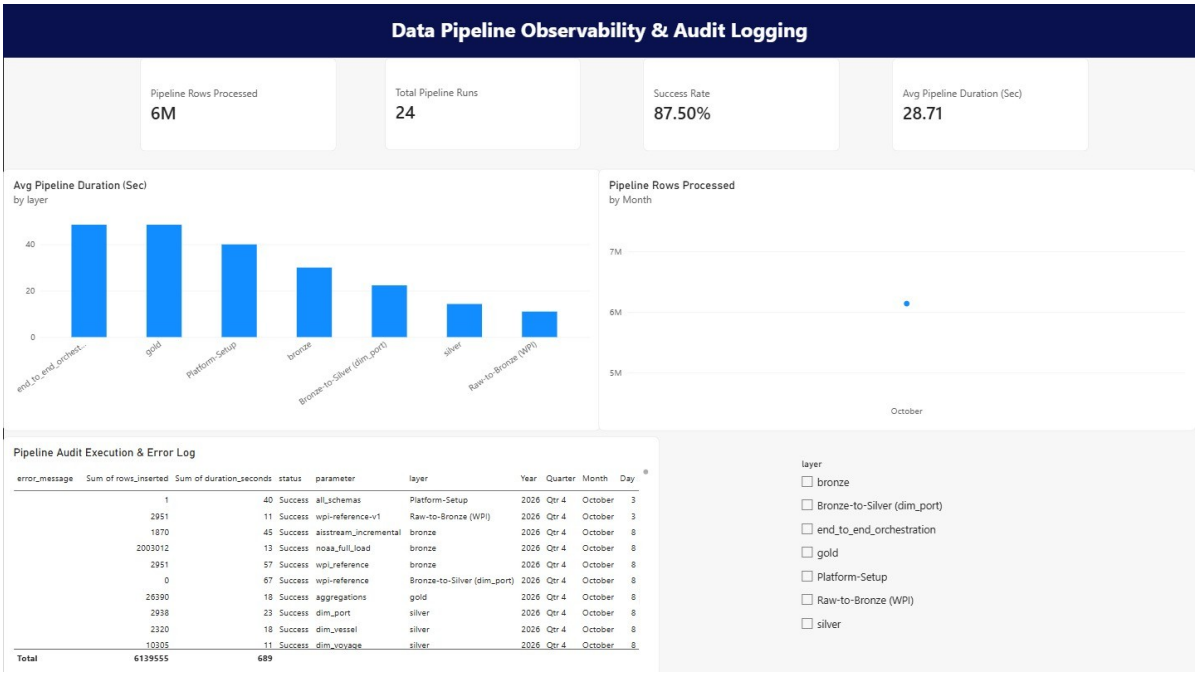


Figure: Power BI Dashboard Page 3: Pipeline Observability & Audit Governance. *Data engineering health dashboard tracking pipeline execution logs, stage runtimes, row ingestion throughput, and quarantine error status.*

9. Conclusion & Production Readiness Evaluation

The Maritime Shipment & Cargo Tracking Lakehouse demonstrates a robust, production-grade architecture that successfully bridges data engineering and business intelligence. By enforcing strict schema validation at Bronze, idempotent MERGE logic at Silver, and a fully optimized Star Schema at Gold, the system achieves zero duplicate data anomalies, sub-second analytical query performance, and comprehensive operational observability.

The technical defense, Spark DAG analysis, and FinOps evaluation presented in this report validate that the platform is computationally efficient, economically scalable, and ready for enterprise operational deployment.